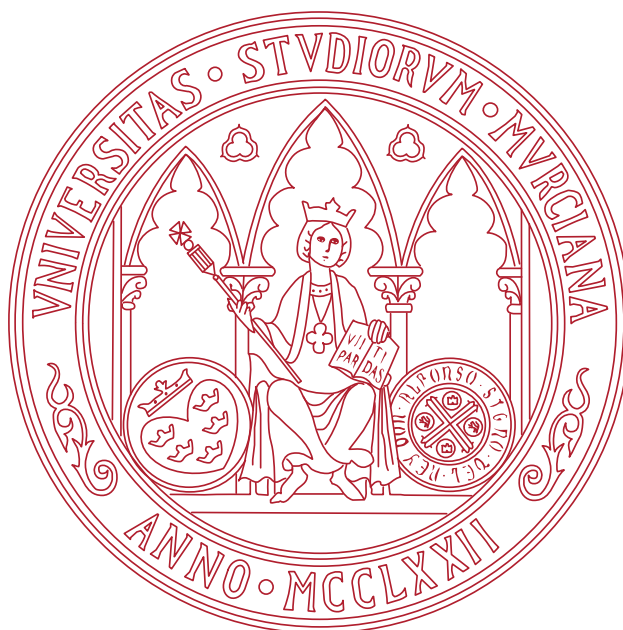




UNIVERSIDAD DE MURCIA

TRABAJO FIN DE GRADO

Impacto de los sistemas de memoria en grandes modelos de lenguaje

Estudiante:

Javier Patricio LUJÁN ROMERO
jp.lujanromero@um.es

Tutor:

Eduardo MARTÍNEZ GRACIÁ
edumart@um.es

Enero 2026

A mi familia por su apoyo incondicional desde el inicio.

Mi madre y su ternura.

Mi padre y su sacrificio.

Índice general

Resumen	3
Extended abstract	4
1. Introducción	5
1.1. Motivación	5
1.2. Objetivos	6
1.3. Estructura del documento	6
2. Estado del arte	7
2.1. Grandes modelos de lenguaje	7
2.2. La ventana de contexto y sus problema	9
2.3. Sistemas de memoria	10
2.3.1. Mem0	11
3. Análisis de objetivos y metodología	13
3.1. Objetivos	13
3.2. Herramientas y tecnologías	13
3.3. Metodología de trabajo	13
4. Diseño y resolución	14
5. Conclusiones y vías futuras	15
Bibliografía	17

Índice de figuras

2.1. Arquitectura de un Transformer. Extraído de [9].	8
2.2. Ejemplo de tokenización.	9
2.3. Pipeline de Mem0. Extraído de [1].	12

Resumen

En este proyecto se plantea el problema de...

Extended abstract

This project faces the problem of...

Introducción

1.1. Motivación

En los últimos años la IA, en particular los grandes modelos de lenguaje – LLMs, por sus siglas en inglés –, han supuesto una revolución y un cambio de paradigma ya no solo dentro de la industria del Machine Learning, sino en todos los ámbitos de la sociedad. Las aplicaciones de chat basadas en estos grandes modelos de texto, como ChatGPT o Google Gemini, se han convertido en herramientas principales para millones de usuarios en todo el mundo realizando una inmensidad de tareas distintas.

Estas tareas tienen diferentes niveles de complejidad, desde tareas simples que se resuelven con una respuesta directa, a tareas más complejas en las que el propio modelo debe replantear los conceptos con los que está trabajando y como los ha de usar para llegar a la solución.

Dentro de estas tareas complejas, se pueden plantear dos situaciones: Una en la que el modelo dispone de toda la información necesaria para resolver la tarea en el prompt, y la complejidad radica en el proceso deductivo que el modelo debe realizar llevarla a cabo exitosamente, como por ejemplo la resolución de problemas matemáticos avanzados. Y, por otro lado, tenemos el caso en el que la resolución exitosa de la tarea requiere de otra información adicional que no se encuentra en el prompt, como ejemplo de este tipo de tarea tenemos la modificación de un código fuente ajustándose a los estándares del proyecto en el que se encuentra.

Es dentro de este segundo tipo de tareas donde ha proliferado el uso de los conocidos agentes de IA: sistemas autónomos capaces de percibir su entorno, razonar sobre posibles cursos de acción, y tomar y ejecutar decisiones [6]. Estos agentes tienen como base un LLM principal el cual realiza el razonamiento, y se le dota de un conjunto diverso de herramientas que le permiten interactuar con su entorno. Ejemplos de estas herramientas son: navegadores web, RAG en bases de datos, editores de código, e incluso otros modelos de IA especializados en diferentes tareas.

Pero para el desarrollo de este trabajo, nos vamos a centrar en un tipo de herramienta en concreto: los sistemas de memoria. Estos sistemas permiten a los agentes almacenar y recuperar información relevante a lo largo del tiempo, mejorando su capacidad para manejar tareas que requieren conocimiento a largo plazo y reduciendo la necesidad de incluir grandes cantidades de texto en el prompt. De esta manera se construyen agentes más eficientes tanto a nivel computacional como en el uso de tokens, que por ende son más económicos y generan un menor impacto ambiental.

1.2. Objetivos

El objetivo principal del trabajo es realizar un estudio sobre el impacto que tienen los sistemas de memoria en el rendimiento de los LLMs en distintos terminos: latencia, accuracy y consumo de tokens.

Con el fin de realizar este primer objetivo se ha creado una pipeline modular, que permite integrar diferentes sistemas de memoria para LLMs, facilitando la experimentación y evaluación de estos sistemas en diversas tareas y datasets. Cumpliendo de esta manera el segundo objetivo, diseñar un sistema accesible que permita evaluar diferentes sistemas de memoria en LLMs de manera sencilla y adaptable.

1.3. Estructura del documento

El trabajo se ha estructurado de acuerdo con las indicaciones dadas en la guía docente de la asignatura correspondiente al Trabajo de Fin de Grado. Y por tanto, el resto del documento se organiza de la siguiente manera:

- En el capítulo 2 se realiza un estudio del estado del arte relativo a los conceptos tratados en este trabajo, abarcando desde la arquitectura detrás de los LLMs hasta los sistemas de memoria.
- En el capítulo 3 se describen de manera detallada los objetivos del trabajo, se introducen las herramientas y tecnologías empleadas, y se explica la metodología de trabajo seguida.
- En el capítulo 4, diseño y resolución, se expone y explica la implementación realizada de la pipeline que permite realizar la evaluación de los sistemas de memoria.
- Finalmente, en el capítulo 5 se exponen los resultados obtenidos tras la evaluación de un sistema de memoria, sacando conclusiones de los mismos y proponiendo vías de mejora a futuro.

CAPÍTULO 2

Estado del arte

En este capítulo se realiza un estudio del estado del arte relativo a los conceptos sobre los que trata el trabajo. Se inicia con una introducción a los grandes modelos de lenguaje en la sección 2.1, permitiendo entender la arquitectura que tienen detras y sus principales carterísticas, haciendo especial hincapie en la ventana de contexto. En la siguiente sección, la 2.2, se describen los principales problemas que afectan a dicha ventana de contexto. Para acontinuación presentar en la sección 2.3 los sistemas de memoria como posible solución a los problemas mencionados anteriormente. Finalmente, se realiza una explicación de Mem0, el sistema de memoria elegido para realizar el estudio.

2.1. Grandes modelos de lenguaje

Como se ha expuesto en la introducción, la IA y los grandes modelos de lenguaje (LLMs) son, cada vez más, un pilar fundamental en nuestra sociedad. Por tanto es necesario entender, con cierto grado de profundidad, que significa cada uno de estos conceptos y en que se diferencian.

Por un lado, el concepto más general de todos es el de inteligencia artificial (IA). La IA es un campo de estudio de la informática cuyo objetivo es crea maquinas inteligentes capaces de mejorar de manera autonoma explorando y aprendiendo del mundo real [10]. Dentro de este campo encontramos el Aprendizaje Computacional, o Machine Learning, que es la rama que se encarga de la contrucción de algoritmos que a partir de un conjunto de datos de entranamiento, son capaces de aprender patrones, que les permite posteriormente realizar predicciones o tomar decisiones sobre datos nuevos que no han visto antes, es decir, les permite generalizar [8]. Finalmente, es dentro del Machine Learning donde nos encontramos con el Aprendizaje Profundo, o Deep Learning, que se acerca por fin a la concepcion generalizada de la IA.

Este aprendizaje profundo, se basa en redes neuronales: un modelo computacional inspirado en de las neuronas biológicas del cerebro humano, que se comunican a través

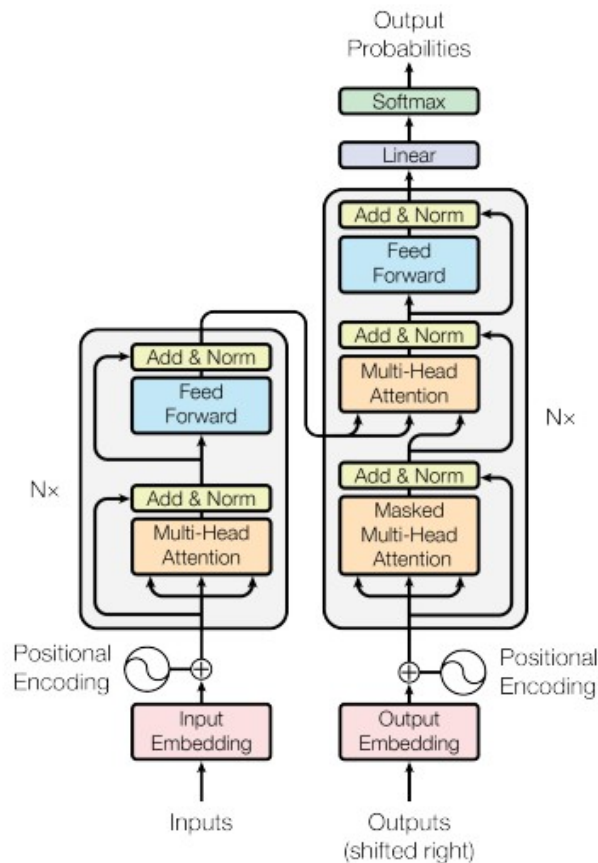


Figura 2.1: Arquitectura de un Transformer. Extraído de [9].

de señales eléctricas [8]. Esto se traduce en una serie de capas conectadas entre sí de perceptrones – un modelo matemático de neurona –. La profundidad radica en el número de capas ocultas, que se encuentran entre la capa de entrada y la de salida, esto les permite aprender representaciones jerárquicas y complejas de los datos, mediante el uso de algún algoritmo que ajusta la fuerza de las conexiones entre las neuronas individuales de capas adyacentes, lo que se conoce como pesos [8].

En un principio entrenar estos modelos de Deep Learning requería recursos computacionales significativos y bastante tiempo. Sin embargo, desde hace unos años se ha desarrollado una nueva arquitectura de red neuronal, los denominados **Transformers**, que ha revolucionado el campo del Deep Learning, especialmente en el procesamiento del lenguaje natural (NLP)[9]. Es gracias a esta Arquitectura que se han podido crear los LLMs (Large Language Models) llamados así por la inmensa cantidad de datos con los que son entrenados y la cantidad de parámetros que tienen.

Cabe por tanto finalizar esta sección explicando brevemente en qué consiste la arquitectura. Un Transformer es una arquitectura de red neuronal basada en el mecanismo de atención, que permite al modelo enfocarse en diferentes partes de la entrada de manera adaptativa, siendo por tanto altamente efectiva a la hora de procesar y generar secuencias de datos, como texto, audio o video; y que al mismo tiempo es altamente paralelizable.



Figura 2.2: Ejemplo de tokenización.

Los Transformers se componen principalmente de dos partes: el codificador (encoder) y el decodificador (decoder), la parte izquierda y derecha de la figura 2.1 respectivamente [9]. El codificador toma la secuencia de entrada previamente convertida en embeddings, representaciones numéricas, y la transforma en una representación interna que captura tanto la palabra como su contexto, mientras que el decodificador utiliza esta representación para generar la secuencia de salida, palabra por palabra de manera auto-regresiva, es decir, utilizando la salida generada previamente para generar la siguiente. Ambos componentes están formados por múltiples capas de atención y redes neuronales feedforward.

2.2. La ventana de contexto y sus problema

En la sección anterior se ha explicado brevemente la arquitectura detrás de los Transformers; sin embargo, no se ha detallado cómo se introduce la secuencia de entrada en estos modelos. Es importante destacar que los modelos no trabajan con texto plano, sino que, como se ha indicado, toman como entrada *embeddings*. Estos *embeddings* son vectores de valores continuos que representan tanto a las palabras como su relación semántica con otras, y son las unidades matemáticas con las que opera el modelo [4].

Para llegar a generar estos *embeddings*, existe un paso previo fundamental: la **tokenización**. En este proceso, las palabras, caracteres individuales o subconjuntos de caracteres, son divididos en unidades mínimas de información llamadas *tokens*[11], tal como se muestra en la figura 2.2. Finalmente, cada uno de estos *tokens* es convertido en un primer *embedding* base para ser procesado por el modelo.

Los tokens constituyen, por tanto, la unidad básica de información con la que se introduce el texto en los modelos de lenguaje. Y al ser la base sobre la que se calcula la representación interna del texto, se utilizan como métrica estándar para establecer los límites de capacidad en la entrada de datos.

Este límite se denomina de manera formal como la *ventana de contexto*, y se refiere a la cantidad máxima de tokens que un modelo puede procesar en una sola instancia. La longitud de la ventana varía significativamente entre arquitecturas: desde modelos

más contenidos como las primeras versiones de LLaMA (4-8K tokens), pasando por estándares actuales como GPT-5 (400K tokens), hasta modelos especializados en contextos masivos como Gemini 2.5 Pro, capaz de procesar hasta un millón de tokens.

Aunque la tendencia actual de la insustria es un progresivo aumento de la ventana de contexto, no se debe confiar exclusivamente en esta como unico modelo para la gestión de la información, pues esta solución no esta exenta de problemas, entre los que destacan los siguientes:

1. Un primer problema que tiene la ventana de contexto es que, como indica su propia definición, es, de antemano, un limite fijado[1]. Esto implica que, aunque se pueda aumentar la ventana de contexto, siempre existirá un limite máximo de tokens que el modelo pueda procesar, lo que puede resultar insuficiente para tareas que requieren trabajar con grandes cantidades de información o con conocimiento a largo plazo.
2. Por otro lado, aunque el contexto con el que se esta trabajando quepa dentro de la ventana, procesar grandes cantidades de texto tiene un coste computacional significativo, lo que se traduce en un coste económico elevado [3]. Esto hace que, aunque se pueda aumentar la ventana de contexto, no sea una solución viable a largo plazo, ni para todas las aplicaciones, pues el coste de procesar grandes cantidades de texto deja de ser economicamente viable.
3. Ademas, aun ignorando tanto el coste computacional como el limite de tokens, la ventana de contexto tiene otra desventaja crucial: El rendimiento de los modelos de lenguaje tiende a degradarse a medida que aumenta el tamaño de la entrada [2]. Siendo aun más pronunciada esta degradación cuando en el contexto se incluye información irrelevante para la tarea especifica que se esta realizando, lo que puede resultar en una disminución significativa del rendimiento del modelo [7].
4. Finalmente, el ultimo desafio que se presenta, es en la propia naturaleza del aprendizaje. La ventana de contexto permite retener información con la que trabajar, similar a la memoria de trabajo humana, pero es incapaz de procesar la información a lo largo del tiempo, e incluso de diferentes conversaciones y chats [5]. Lo que limita las capacidades de un LLM para generalizar y aprender de la experiencia, pues no puede retener información a largo plazo, lo que se traduce en una falta de continuidad y coherencia en las conversaciones, y en una incapacidad para aprender de la experiencia pasada.

2.3. Sistemas de memoria

Como solución a estos problemas, se ha propuesto diferentes enfoques, entre los que destacan los sistemas de memoria, cuyo objetivo es imitar la memoria a largo plazo, permitiendo a los modelos *recordar* la información imporante en el momento adecuado. Reducienndo asi la necesiad de manejar grandes cantidades de texto. Economizando, e incluso mejorando, el rendimiento de los modelos en tareas que requieren trabajar con conocimiento a largo plazo.

Para que estos sistemas de memoria sean efectivos, se busca que cumplan ciertas características, entre las que destacan:

1. Selección adecuada de la información relevante para almacenar en memoria.
2. Consolidación efectiva de la información almacenada.
3. Recuperación eficiente de la información cuando sea necesario.
4. Capacidad de adaptación y actualización de la memoria con nueva información.

2.3.1. Mem0

Entre los diferentes sistemas de memoria que hay en el mercado, Mem0 [1] ha sido elegido como base para este trabajo, permitiendo realizar la comparativa entre la ventana de contexto y el uso de un sistema de memoria. Esta decisión se ha tomado así por ser Mem0 un ejemplo canónico de sistema de memoria, por su facilidad de uso e integración, y por ser un sistema actual y actualizado.

Es por ende necesario entender el funcionamiento de Mem0 para poder comprender la relación entre la mejora que supone y los posibles factores negativos que se pueden asociar a este.

El componente principal de Mem0 es una base de datos vectorial en la que se almacenarán las memorias conforme se vayan generando. Mem0 actúa de wrapper sobre esta base de datos, permitiendo usar cualquier tipo de bdd vectorial que se desee, lo que aporta una gran flexibilidad y, en caso de que sea necesario, privacidad.

Su comportamiento se divide en dos fases principales: la extracción de posibles memorias de la conversación, y la actualización de la base de datos:

La **extracción** como se aprecia en la figura 2.3, se realiza en cada iteración del chat, tomando como entrada el último par de mensajes –usuario y respuesta del modelo–; una secuencia con los últimos m mensajes, siendo este número un hiperparámetro, y finalmente un resumen de la conversación generado de manera asincrónica en la base de datos. Toda esta información se introduce en un prompt y se envía a un LLM, previamente configurado por el operador del sistema, con el fin de que este extraiga memorias candidatas para añadir a la base de datos.

Una vez realizado este proceso se procede con la **actualización**. En esta fase se realiza el mismo procedimiento para cada memoria candidata extraída en el paso anterior que consiste en realizar una búsqueda semántica en la base de datos tomando como prompt la memoria candidata. Entonces, usando tanto la memoria candidata como las memorias obtenidas en la consulta, se realiza una segunda llamada a un LLM el cual debe decidir entre cuatro posibles acciones: *ADD*, *UPDATE*, *DELETE* y *NOOP*. Que significan:

- **NOOP**: No hacer nada, la memoria candidata no aporta nada nuevo a la base de datos.

- **UPDATE**: Actualizar alguna de las memorias obtenidas en la consulta con la información de la memoria candidata.
- **ADD**: Añadir la memoria candidata como una nueva memoria en la base de datos.
- **DELETE**: Borrar alguna de las memorias obtenidas en la consulta, pues ya no es relevante.

En la figura 2.3 se puede observar toda la pipeline de Mem0. (Termina de explicar que gracias a esto se obtiene lo buscado)

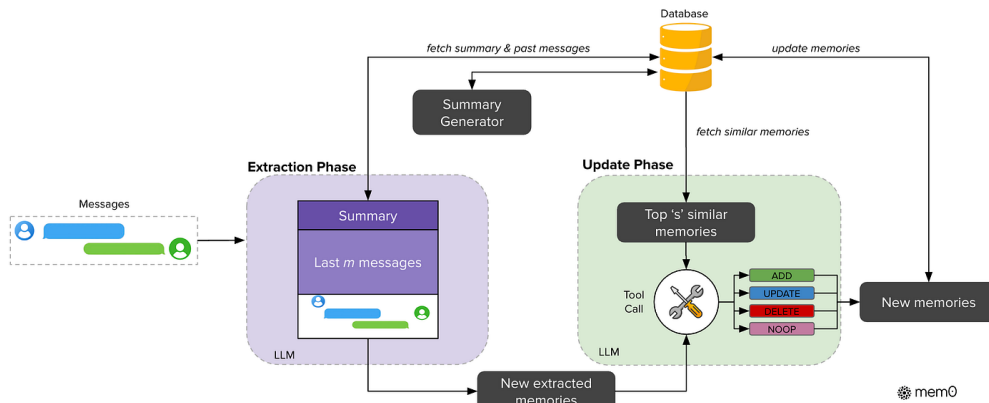


Figura 2.3: Pipeline de Mem0. Extraído de [1].

CAPÍTULO 3

Análisis de objetivos y metodología

Una vez tratado el estado del arte, realizamos en este capítulo el análisis de los objetivos del mismo, estableciendo los elementos concretos que nos van a permitir llevar a cabo dichos objetivos así como la metodología de desarrollo.

3.1. Objetivos

3.2. Herramientas y tecnologías

3.3. Metodología de trabajo

CAPÍTULO 4

Diseño y resolución

En este capítulo se realiza la descripción de la implementación de...

CAPÍTULO 5

Conclusiones y vías futuras

Finalizamos el trabajo estableciendo las conclusiones y vías futuras...

Bibliografía

- [1] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [2] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts llm performance. Technical report, Chroma, July 2025. URL: <https://research.trychroma.com/context-rot>.
- [3] Kyle Montgomery, David Park, Jianhong Tu, Michael Bendersky, Beliz Gunel, Dawn Song, and Chenguang Wang. Predicting task performance with context-aware scaling laws. *arXiv preprint arXiv:2510.14919*, 2025.
- [4] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. Text and code embeddings by contrastive pre-training. arxiv 2022. *arXiv preprint arXiv:2201.10005*, 2022.
- [5] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: Towards llms as operating systems. 2023.
- [6] Xiaodong Qu, Andrews Damoah, Joshua Sherwood, Peiyan Liu, Christian Shun Jin, Lulu Chen, Minjie Shen, Nawwaf Aleisa, Zeyuan Hou, Chenyu Zhang, et al. A comprehensive review of ai agents: Transforming possibilities in technology and beyond. *arXiv preprint arXiv:2508.11957*, 2025.
- [7] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pages 31210–31227. PMLR, 2023.
- [8] Cole Stryker, Fangfang Lee, Dave Bergmann, and Mark Scapicchio. The 2026 Guide to Machine Learning. IBM Think, 2026. Accedido: 04-02-2026. URL: <https://www.ibm.com/think/machine-learning>.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

- [10] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL: <https://arxiv.org/abs/2404.13501>, arXiv:2404.13501.
- [11] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.